

今天 上午1:26

如何將 <https://github.com/firefirelu-create/ally-project/tree/main/job-1> 資料更新到本機

可以。你提供的網址是 GitHub 的：

```
https://github.com/firefirelu-create/ally-project/tree/main/job-1
```

這代表 `ally-project` 是整個 **GitHub Repository**，而 `job-1` 是裡面的一個資料夾。

如果你的目標是：

把 GitHub 上最新的 `job-1` 資料更新到自己的電腦

最建議使用 Git，不要每次重新下載 ZIP。GitHub 官方也建議：第一次用 `clone`，之後用

`git pull` 更新。  [GitHub Docs](#) +1

情況一：你的電腦「已經有」這個專案

假設你本機已經有：

```
ally-project
└─ job-1
```

先在 **VS Code Terminal** 開啟 `ally-project` 資料夾。

輸入：

Bash

```
git status
```

如果看到類似：

```
On branch main
Your branch is up to date with 'origin/main'.
```

接著直接：

Bash

```
git pull origin main
```

這樣 Git 就會把 GitHub 最新版本同步到本機。 `git pull` 本身就是先抓取遠端更新，再合併到目前分支。  [GitHub Docs](#)

情況二：你的電腦「還沒有」這個專案

例如你想放在：

```
D:\VibeCoding
```

先在 Terminal 輸入：

Bash

```
cd D:\VibeCoding
```

然後：

Bash

```
git clone https://github.com/firefirelu-create/ally-project.git
```

完成後會產生：

```
D:\VibeCoding
├─ ally-project
│   └─ job-1
│   └─ ...
```

這是比較標準的做法； `git clone` 會把整個 Repository 複製到本機。  [GitHub](#)

然後：

Bash

```
cd ally-project
```

再輸入：

Bash

```
git status
```

之後 GitHub 有更新時，只需要：

```
Bash
```

```
git pull
```

就可以了。  GitHub

如果你「只想更新 job-1」

這點要特別注意：

Git 的正常 `pull` 是更新 整個 `ally-project` **Repository**，不是只更新 `job-1`。所以如果 GitHub 上其他資料夾也有變動，也可能一起同步。

如果你的需求其實是：

我本機已經有 `ally-project`，只想把 GitHub 最新的 `job-1` 拉下來，不想影響其他資料。

可以另外處理，我可以依照你目前的資料夾狀況教你。

如果你是用 VS Code，我建議你這樣做

在 VS Code：

Terminal → New Terminal

先輸入：

```
Bash
```

```
git status
```

把畫面結果貼給我。

我可以直接告訴你下一步該輸入：

```
Bash
```

```
git pull
```

還是需要先設定：

```
Bash
```

```
git remote
```

或其他指令。

尤其如果你現在本機的 `job-1` 已經有自己修改過的 HTML/CSS/JS，我建議先不要直接 `git pull`，先確認狀態，避免把你目前的程式弄亂。

```
已經pull但是本機還是沒有更新到和github上一樣
From    firefirelu-create/ally-project
* branch      main    -> FETCH_HEAD
Already up to date.
```

這個訊息其實很重要：

```
From https://github.com/firefirelu-create/ally-project
* branch      main -> FETCH_HEAD
Already up to date.
```

意思是 **Git** 認為你的本機 `main` 已經和 **GitHub** 的 `main` 一樣。

但你看到 GitHub 的 `job-1` 和本機不一樣，通常不是 `pull` 本身的問題，而是下面幾種情況之一：

1. 你目前開的本機資料夾不是這個 Git Repository。
2. 本機目前不是 `main` 分支。
3. GitHub 網頁上的 `job-1` 有檔案，但那些檔案沒有被 Git 追蹤。
4. 你本機有修改，或 `.gitignore` 排除了某些檔案。
5. 你看的 **GitHub** `job-1` 和本機的 `job-1` 其實不是同一個位置。

我們不要猜，直接一步一步確認。

第一步：在「你目前的本機 `job-1` 資料夾」開 Terminal

輸入：

Bash

```
git remote -v
```

應該會看到：

```
origin  https://github.com/firefirelu-create/ally-project.git (fetch)
origin  https://github.com/firefirelu-create/ally-project.git (push)
```

接著輸入：

Bash

```
git branch
```

應該看到：

```
* main
```

再輸入：

Bash

```
git status
```

第二步：確認 GitHub 最新內容

輸入：

Bash

```
git fetch origin
```

然後：

Bash

```
git log --oneline --all -5
```

這會列出最近 5 次 Commit。

第三步：最重要的一個檢查

請輸入：

Bash

```
git diff main origin/main
```

如果什麼都沒有顯示：

代表 Git 判斷本機 `main` 和 GitHub `origin/main` 完全相同。

如果有很多內容出現，例如：

```
diff --git ...
```

那就表示本機和 GitHub 確實不同。

如果你的目標是「直接讓本機變成 GitHub 現在的樣子」

⚠ 如果本機 `job-1` 裡面有你自己修改、而且還沒上傳 GitHub 的程式，先不要做這一步。

如果你確定：

本機的東西不要了，我就是要完全以 GitHub 為準。

可以執行：

Bash

```
git fetch origin  
git reset --hard origin/main
```

然後：

Bash

```
git clean -fd
```

最後：

Bash

```
git status
```

這樣本機的 Git 專案就會被強制調整成 GitHub `main` 的版本。

不過，我現在比較懷疑你的問題是「資料夾位置」

因為你給我的 GitHub 網址是：

```
ally-project/tree/main/job-1
```

這代表：

```
ally-project
└─ job-1
```

`job-1` 本身不是 Repository 。

如果你是在 `job-1` 裡面操作 Git，情況可能會和你想像的不一樣。

請你現在先不要再輸入其他指令

在你「目前本機的 `job-1` 資料夾」Terminal 輸入這 4 個：

Bash

```
pwd
```

Bash

```
git remote -v
```

Bash

```
git branch
```

Bash

```
git status
```

把這 4 個結果完整貼給我。

我可以直接從結果判斷你現在是不是在正確的 `ally-project`、是不是 `main`，以及為什麼明明 `Already up to date`，本機畫面卻和 GitHub 的 `job-1` 不一樣。

```
PS C:\Users\USER\Desktop\ally-project> pwd
```

```
Path
```

```
C:\Users\USER\Desktop\ally-project
```

```
PS C:\Users\USER\Desktop\ally-project> git remote -
v(fetch)(push)
```

```
PS C:\Users\USER\Desktop\ally-project> git branch
* main
PS C:\Users\USER\Desktop\ally-project> git status
On branch main
顯示更多
```

請幫我判斷為什麼本機 ally-project 和 GitHub 的 job-1 不一致。

本機路徑：C:\Users\USER\Desktop\ally-project

```
git remote -v :
origin https://github.com/firefirelu-create/ally-project.git
(fetch)
origin https://github.com/firefirelu-create/ally-project.git
(push)
PS C:\Users\USER\Desktop\ally-project>
顯示更多
```

我已經可以幫你縮小問題範圍了。

你目前的 Git 狀態是：

```
HEAD -> main
origin/main
origin/HEAD
```

全部都指向：

```
6e157b0 feat:標題
```

所以可以確定：

GitHub 的 main 和你本機 ally-project 的 main，目前是同一個 Commit。

因此，問題不是 `git pull` 沒有成功。

更可能是你看到的「GitHub job-1」與本機檔案的位置或內容不是 Git 所追蹤的同一份資料。

我們現在直接查 `job-1`

請在：

```
C:\Users\USER\Desktop\ally-project
```

執行下面這個指令：

PowerShell

```
Get-ChildItem
```

看看本機是否真的有：

```
job-1
```

如果有，再執行：

PowerShell

```
Get-ChildItem .\job-1
```

這會列出本機 `job-1` 裡面的檔案。

接著做最重要的檢查

請輸入：

PowerShell

```
git ls-tree -r --name-only origin/main
```

這個指令會直接告訴我們：

GitHub 的 `main` 目前實際追蹤了哪些檔案。

你應該會看到類似：

```
job-1/index.html
job-1/xxx.html
job-1/css/xxx.css
...
```

如果看到 `job-1/index.html`

例如結果有：

```
job-1/index.html
```

那我們再檢查 GitHub 版本的內容：

PowerShell

```
git show origin/main:job-1/index.html
```

這會直接把 **GitHub main** 裡面的 `job-1/index.html` 顯示出來。

而本機版本則可以用：

PowerShell

```
Get-Content .\job-1\index.html
```

這樣就可以非常準確地比較兩份。

還有一個非常重要的可能性

你現在的 Commit 是：

```
6e157b0 feat:標題
```

而你說 GitHub 網頁：

```
https://github.com/firefirelu-create/ally-project/tree/main/job-1
```

看起來和本機不同。

這時候我要特別確認：

GitHub 網頁上 `job-1` 的檔案，是否真的已經包含在 `6e157b0` ？

執行：

PowerShell

```
git show --stat 6e157b0
```

它會告訴我們：

```
6e157b0 feat:標題
```

```
job-1/xxx.html | ...  
job-1/xxx.css  | ...
```

到底這個 Commit 改了哪些檔案。

目前不要執行 `reset --hard`

先不要執行：

PowerShell

```
git reset --hard origin/main
```

因為目前結果已經證明：

```
本機 main = GitHub origin/main
```

所以 `reset` 很可能也不會解決問題。

請你現在只做這 3 個指令

按照順序貼給我：

①

PowerShell

```
Get-ChildItem
```

②

PowerShell

```
Get-ChildItem .\job-1
```

③

PowerShell

```
git ls-tree -r --name-only origin/main
```

尤其是第③個最重要。

我可以從這三個結果直接判斷，到底是：

A. `job-1` 根本不在目前 Git 版本裡

或

B. `job-1` 在 Git 裡，但本機資料夾內容不同

或

C. GitHub 網頁看的其實不是目前 `main` 的內容

或

D. VS Code 開的不是 `C:\Users\USER\Desktop\ally-project`

你現在的 Git 本身看起來是正常的，所以我們不用重灌 Git，也不用重新 clone；先把 `job-1` 的實際 Git 狀態查出來即可。

目錄: `C:\Users\USER\Desktop\ally-project`

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d----	2026/9/20 下午 03:02		css
d----	2026/9/19 上午 02:39		image
d----	2026/9/19 上午 02:39		JavaScrip
d----	2026/9/23 上午 01:23		job-1
-a----	2026/9/20 下午 03:30	5040	index.html
-a----	2026/9/23 上午 01:23	136704	工作流程-20

顯示更多

很好，這個結果已經發現一個很重要的線索。

你本機現在確實有：

```
C:\Users\USER\Desktop\ally-project
|
├─ css
├─ image
├─ JavaScrip
├─ job-1      ← 有這個資料夾
```

```
└─ index.html
└─ 工作流程-2026.09.16.doc
└─ 表單練習.HTML
```

而且 `job-1` 的修改時間是：

2026/9/23 上午 01:23

這代表 `job-1` 這個資料夾確實存在。

但是現在還不能確定它是不是 GitHub 上那個 `job-1`。

請做下一步

在目前這個位置：

```
PS C:\Users\USER\Desktop\ally-project>
```

輸入：

PowerShell

```
Get-ChildItem .\job-1
```

把結果完整貼給我。

接著再輸入這個：

PowerShell

```
git ls-tree -r --name-only origin/main
```

這個結果非常重要。

它會告訴我們 **GitHub** `main` 實際追蹤的檔案。

例如如果出現：

```
job-1/index.html
job-1/style.css
job-1/script.js
```

就表示 GitHub 確實有這些檔案。

如果完全沒有：

```
job-1/...
```

那我們就會找到原因：

你的本機有 `job-1`，但這些檔案可能沒有被 **Git** 納入追蹤，因此 `git pull` 不會下載它們。

這也可以解釋為什麼你一直看到：

```
Already up to date.
```

因為 **Git** 只會同步 **GitHub Repository** 裡被 **Commit** 的內容，不會因為本機存在一個資料夾就自動把它當成 **GitHub** 的內容。

所以現在先不要再 `pull`、也不要 `reset`

請只貼這兩個結果：

PowerShell

```
Get-ChildItem .\job-1
```

以及：

PowerShell

```
git ls-tree -r --name-only origin/main
```

我看完就可以直接告訴你為什麼 **GitHub** 的 `job-1` 和本機不一致，以及下一步該怎麼修正。

目錄: C:\Users\USER\Desktop\ally-project\job-1

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	2026/9/20 下午 03:05	1196	job-1.html
-a----	2026/9/20 上午 06:00	13495	mountant-ge mini.html

-a---- 2026/9/23 上午 01:23 34816 mountant.xl

S

顯示更多

很好，現在問題更清楚了。

你本機的 `job-1` 有：

```
job-1/  
├─ job-1.html  
├─ mountant-gemini.html  
└─ mountant.xls
```

其中兩個檔案的時間是 **9/20**，只有 `mountant.xls` 是 **9/23 01:23**。

但是我們還缺最後一個關鍵結果：**GitHub** 的 `main` 到底有沒有追蹤這三個檔案。

請在：

```
PS C:\Users\USER\Desktop\ally-project>
```

執行：

PowerShell

```
git ls-tree -r --name-only origin/main
```

如果結果有：

```
job-1/job-1.html  
job-1/mountant-gemini.html  
job-1/mountant.xls
```

那代表 GitHub 和本機的 Git 版本確實都有這三個檔案。

如果 GitHub 網頁看到的內容仍然不同，我們下一步就直接比較檔案內容。

如果結果沒有 `job-1/...`

例如只看到：

```
css/...  
image/...
```

而沒有：

```
job-1/
```

那原因就找到了：

本機有 `job-1`，但 GitHub `main` 的 Git 版本沒有追蹤這個資料夾。

這種情況下，`git pull` 當然不會把 GitHub 網頁上的 `job-1` 更新到本機，因為那個 `job-1` 很可能是另一個版本、另一個分支，或者你在 GitHub 網頁上看到的內容並沒有進入目前 `main` 的 Commit。

請再做一個非常重要的確認

直接執行：

PowerShell

```
git status --ignored
```

然後把結果貼給我。

這可以檢查 `job-1` 裡面的檔案是不是被 `.gitignore` 忽略。

所以現在請貼兩個結果

①

PowerShell

```
git ls-tree -r --name-only origin/main
```

②

PowerShell

```
git status --ignored
```

有了這兩個結果，我就可以直接判斷問題，不需要再一直嘗試 `git pull`。

origin/main

JavaScript/2026.09.01/2026.09.01-1.html

JavaScript/2026.09.01/2026.09.01-10.html

JavaScript/2026.09.01/2026.09.01-2.html

JavaScript/2026.09.01/2026.09.01-3.html

JavaScript/2026.09.01/2026.09.01-4.html

JavaScript/2026.09.01/2026.09.01-5.html

JavaScript/2026.09.01/2026.09.01-6.html

JavaScript/2026.09.01/2026.09.01-7.html

JavaScript/2026.09.01/2026.09.01-8.html

JavaScript/2026.09.01/2026.09.01-9.html

顯示更多

太好了，現在已經可以確定原因了。

你剛才的結果顯示：

job-1/job-1.html

job-1/mountant-gemini.html

job-1/mountant.xls

確實存在於：

origin/main

也就是說：

GitHub 的 `job-1` 確實已經在 `main` 裡，而且你的本機 **Git** 也已經同步到同一個 **Commit** `6e157b0`。

所以現在問題不是 GitHub 沒更新，也不是 `git pull` 失敗。

真正值得注意的是這個

你本機：

job-1/job-1.html

修改時間：

2026/9/20 下午 03:05

job-1/mountant-gemini.html

修改時間：

2026/9/20 上午 06:00

但：

job-1/mountant.xls

修改時間：

2026/9/23 上午 01:23

這表示 本機的檔案時間不能直接拿來判斷是否和 **GitHub** 一樣。

Git 判斷檔案是否更新，是看 **Commit 內容**，不是看 Windows 的「最後修改時間」。

我們現在直接比較「內容」

這才是最準確的方法。

假設你說的是：

GitHub `job-1` 裡面的檔案內容，和本機 `job-1` 裡面的內容不同。

我們直接讓 Git 比較。

請輸入：

PowerShell

```
git diff origin/main -- job-1
```

如果沒有任何輸出

代表：

本機 `job-1` 的內容與 GitHub `main` 的 `job-1` 完全相同。

那麼你看到的不一致，很可能是：

- VS Code 開到另一個資料夾
 - 瀏覽器開的是舊檔案
 - GitHub 網頁看的不是 `main`
 - 檔案顯示/快取問題
-

如果有大量內容

例如：

```
diff --git a/job-1/job-1.html b/job-1/job-1.html
...
```

那才代表：

本機 `job-1` 與 GitHub `job-1` 的內容真的不同。

這時候我就可以告訴你如何安全地把本機 `job-1` 更新成 GitHub 版本。

另外一個很重要的檢查

請輸入：

PowerShell

```
git status --short
```

如果沒有任何東西：

```
PS C:\Users\USER\Desktop\ally-project>
```

就代表本機沒有 Git 未提交修改。

你現在只需要做這一個

先輸入：

PowerShell

```
git diff origin/main -- job-1
```

把結果完整貼給我。

如果完全沒有任何文字，也請告訴我「沒有輸出」。

我就能進一步判斷：到底是 檔案內容真的不同，還是其實 **GitHub** 和本機已經完全一致，只是你目前看到的畫面不是同一份檔案。

```
git status --ignored
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

這個結果非常關鍵。現在我們可以確定：

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

再加上你前面提供的：

```
6e157b0 (HEAD -> main, origin/main, origin/HEAD) feat:標題
```

以及：

```
job-1/job-1.html
job-1/mountant-gemini.html
job-1/mountant.xls
```

結論

你的本機 **Git** 與 **GitHub** `main` 已經是同步的。

所以現在不要再做 `git pull`，也不要 `reset --hard`。

真正需要確認的是：

你認為「不一致」的到底是哪一個檔案，以及內容差在哪裡。

最直接の確認方法

請在 PowerShell 執行：

PowerShell

```
git diff origin/main -- job-1
```

情況 A：完全沒有任何輸出

如果畫面直接回到：

```
PS C:\Users\USER\Desktop\ally-project>
```

那就代表：

本機 `job-1` 和 GitHub `main` 裡的 `job-1` 內容完全相同。

這時候問題就不是 Git，而比較可能是你：

- 在 VS Code 開到另一個 `ally-project`
- 瀏覽器開到舊的 HTML
- GitHub 看的是不同分支
- Windows / VS Code 顯示的是舊內容

情況 B：出現 `diff`

例如：

```
diff --git a/job-1/job-1.html b/job-1/job-1.html
```

那才表示：

本機和 GitHub 的 `job-1` 真的有內容差異。

我建議你再做一個最簡單的確認

直接執行：

PowerShell

```
git show origin/main:job-1/job-1.html
```

這會顯示 GitHub `main` 裡的 `job-1.html` 真正內容。

然後再執行：

PowerShell

```
Get-Content .\job-1\job-1.html
```

這是本機 `job-1.html` 的內容。

兩者如果完全一樣，就證明同步沒有問題。

★ 但如果你的真正需求是：

「我不管原因，我就是要讓 `C:\Users\USER\Desktop\ally-project\job-1` 變成 GitHub 網頁目前看到的樣子。」

那我可以給你一套安全的「GitHub → 本機強制同步 `job-1`」指令，而且只處理 `job-1`，不會動到你其他的：

```
css
image
JavaScript
index.html
```

如果你就是要這個，我可以直接一步一步帶你做。

我不管原因，我就是要讓 `C:\Users\USER\Desktop\ally-project\job-1` 變成 GitHub 網頁目前看到的樣子。

可以。既然你的目標很明確：

以 GitHub `main/job-1` 為準，讓本機 `C:\Users\USER\Desktop\ally-project\job-1` 直接變成 GitHub 上的版本。

而且你目前已經確認 GitHub 與本機都是 `6e157b0`，我們可以只處理 `job-1`，不動其他資料夾。

⚠ 先注意

下面的方法會把本機 `job-1` 裡面與 GitHub 不同的檔案內容覆蓋掉。

如果本機 `job-1` 有你想保留的修改，先不要執行。

方法一：直接從 GitHub 還原 `job-1` ★推薦

你現在的位置已經是：

```
C:\Users\USER\Desktop\ally-project
```

直接執行：

PowerShell

```
git restore --source=origin/main -- job-1
```

然後執行：

PowerShell

```
git status
```

如果顯示：

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

就完成了。

但是有一個重要問題

`git restore` 只會處理 Git 已經追蹤的檔案。

如果你的本機 `job-1` 裡面有 GitHub 沒有的額外檔案，它不會自動刪除。

例如本機：

```
job-1
├─ job-1.html
├─ mountant-gemini.html
├─ mountant.xls
└─ test.html      ← GitHub 沒有
```

`test.html` 可能還會留著。

如果你要「完全一模一樣」

也就是：

GitHub `job-1` 有什麼，本機就有什麼；GitHub 沒有什麼，本機 `job-1` 就不要再有。

那麼我建議用下面這個方式。

① 先備份目前 `job-1`

在 PowerShell：

PowerShell

```
Copy-Item .\job-1 .\job-1-backup -Recurse
```

現在會變成：

```
ally-project
├─ job-1
└─ job-1-backup
```

這樣即使有問題，原本檔案還在。

② 還原 GitHub 版本

PowerShell

```
git restore --source=origin/main -- job-1
```

③ 查看是否還有 GitHub 沒有的檔案

PowerShell

```
git clean -nd -- job-1
```

這一步非常重要。

它只會「預覽」要刪除什麼，不會真的刪除。

如果看到例如：

```
Would remove job-1/test.html
Would remove job-1/abc.txt
```

代表這些是本機多出來、Git 沒有追蹤的檔案。

④ 確定真的要刪除才執行

PowerShell

```
git clean -fd -- job-1
```

完成後：

PowerShell

```
git status
```

★ 我更建議你現在這樣做

因為你只是想讓 `job-1` 跟 GitHub 一樣，我們不要一次執行很多危險指令。

現在先執行這一行：

PowerShell

```
git restore --source=origin/main -- job-1
```

然後：

PowerShell

```
git status
```

把結果貼給我。

如果仍然沒有變化，我再帶你做下一步的「完全同步」，而且我們會保留 `job-1-backup`，不會讓你的原始檔案消失。

On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

這個結果已經可以確定一件事：

Git 認定你本機 `job-1` **和 GitHub** `main/job-1` **完全相同。**

所以再執行 `pull` 、 `restore` 都不會改變檔案。

但你明確看到 GitHub 網頁和本機不一樣，因此我們現在要做的不是再同步 Git，而是直接把 GitHub `job-1` 的檔案重新取出來覆蓋本機。

最直接的方法：重新下載 GitHub 的 `job-1`

你目前不要刪除原本的 `job-1`，先改名備份。

在：

```
C:\Users\USER\Desktop\ally-project
```

執行：

PowerShell

```
Rename-Item .\job-1 .\job-1-old
```

現在會變成：

```
ally-project
├─ job-1-old
├─ css
├─ image
├─ JavaScript
└─ ...
```

接著執行：

PowerShell

```
git checkout origin/main -- job-1
```

這會從目前 GitHub `origin/main` 的版本重新建立：

```
ally-project
└─ job-1
    ├─ job-1.html
    ├─ mountant-gemini.html
    └─ mountant.xls
```

然後檢查：

PowerShell

```
Get-ChildItem .\job-1
```

如果你要的是「GitHub 網頁現在看到的內容」

這裡還有一個非常重要的可能性：

你給我的 GitHub 網址是：

```
github.com/firefirelu-create/ally-project/tree/main/job-1
```

而你的 Git 顯示：

```
6e157b0 (HEAD -> main, origin/main, origin/HEAD) feat:標題
```

這表示 **GitHub** `main` 和本機 **Git** 是同一版本。

因此如果你重新建立 `job-1` 後，看到的仍然和 GitHub 網頁不同，幾乎可以確定：

你在 **GitHub** 網頁上看到的檔案內容，並不是你本機目前 **Git** 所取得的那一份內容。

這時我建議不要再猜。

先做這一步

執行：

PowerShell

```
Rename-Item .\job-1 .\job-1-old
```

然後：

PowerShell

```
git checkout origin/main -- job-1
```

再：

PowerShell

```
Get-ChildItem .\job-1
```

把最後一個結果貼給我。

先不要刪除 `job-1-old`。它是你的備份，等確認 GitHub 版本正確後再處理。

我不小心把原本在github上的資料覆蓋掉...下載下來的job-1.html是錯的
正確的版本是2026/9/23 1點前

先別急，還有機會救回來。你現在最重要的是：不要再 `pull`、不要 `push`、不要 `reset -hard`，也不要刪除目前任何備份資料夾。

你說「正確版本是 2026/9/23 凌晨 1 點前」，這表示我們要找的是 **GitHub** 在那個時間之前的某個 **Commit**。GitHub 的 Commit 歷史可以用來找舊版本，並且可以直接比較不同 Commit。 [GitHub Docs](#)

先做一件最重要的事

你之前的 Git 歷史是：

```
6e157b0 feat:標題  
0d7b8a4 feat:大綱  
ae7c98d initial commit
```

所以你現在的 `job-1.html` 很可能可以從較早的 **Commit** 找回。

在：

```
C:\Users\USER\Desktop\ally-project
```

執行：

PowerShell

```
git log --all --oneline --decorate -- job-1/job-1.html
```

這個指令非常重要。

它會告訴我們：

```
| job-1.html 到底在哪些 Commit 被修改過。
```

接著再執行

PowerShell

```
git log --all --format="%h | %ad | %s" --date="iso" -- job-1/job-1.html
```

你可能會看到類似：

```
6e157b0 | 2026-09-23 00:45:xx +0800 | feat:標題  
0d7b8a4 | 2026-09-22 23:xx:xx +0800 | feat:大綱  
ae7c98d | 2026-09-20 ... | initial commit
```

這樣我們就能精確找到 **9/23 凌晨 1 點以前的版本**。

如果你看到 `0d7b8a4` 是正確版本

先不要直接覆蓋。

可以先把舊版本「另外存一份」：

PowerShell

```
git show 0d7b8a4:job-1/job-1.html > .\job-1\job-1-恢復版本.html
```

這樣你會得到：

```
job-1  
├─ job-1.html  
├─ job-1-恢復版本.html  
├─ mountant-gemini.html  
└─ mountant.xls
```

然後可以用 VS Code 打開：

```
job-1-恢復版本.html
```

確認是不是你要的**9/23 凌晨 1 點以前的正確版本**。

最重要

現在先不要執行 `git push` 。

因為如果你剛剛不小心把 GitHub 的正確版本也覆蓋掉，而且已經 push，GitHub `main` 現在可能已經指向錯誤版本。

但只要舊 Commit 還存在，我們通常仍然可以從 Git 歷史找回。

請你現在只執行這一個指令

PowerShell

```
git log --all --format="%h | %ad | %s" --date="iso" -- job-1/job-1.html
```

把完整結果貼給我。

我會直接幫你判斷哪一個 Commit 是 **2026/9/23 凌晨 1 點以前的正確** `job-1.html`，然後給你一個「先備份 → 恢復 → 確認 → 再決定是否上傳 GitHub」的安全步驟。

```
ec2f821 | 2026-09-23 02:09:51 +0800 | feat:job1
060d8e1 | 2026-09-23 02:07:19 +0800 | feat:job1
:
ec2f821 | 2026-09-23 02:09:51 +0800 | feat:job1
060d8e1 | 2026-09-23 02:07:19 +0800 | feat:job1
6e157b0 | 2026-09-20 15:08:22 +0800 | feat:標題
0d7b8a4 | 2026-09-20 09:33:30 +0800 | feat:大綱
```

找到了關鍵點。你提供的紀錄可以確定：

```
ec2f821 | 2026-09-23 02:09:51 | feat:job1
060d8e1 | 2026-09-23 02:07:19 | feat:job1
6e157b0 | 2026-09-20 15:08:22 | feat:標題
0d7b8a4 | 2026-09-20 09:33:30 | feat:大綱
```

這代表什麼？

2026/9/23 凌晨 1:00 前，Git 歷史裡最後一個版本是：

```
6e157b0
2026-09-20 15:08:22
feat:標題
```

而你真正的 `job-1` 版本，是在：

```
060d8e1 02:07
ec2f821 02:09
```

才進入 Git Commit。

所以有一個非常重要的可能：

你原本正確的 `job-1.html` 可能是在 9/23 凌晨 1 點以前存在於本機，但當時尚未 Commit 到 Git。

因此，不能直接認定 `6e157b0` 就是你要的正確版本。

現在先不要動 GitHub

尤其不要：

PowerShell

```
git push
```

也不要刪掉你剛才改名的：

```
job-1-old
```

如果它還存在，它反而可能就是我們要找的正確檔案！

第一步：先查看 `job-1-old`

請輸入：

PowerShell

```
Get-ChildItem .\job-1-old
```

如果看到：

```
job-1.html  
mountant-gemini.html  
mountant.xls
```

那很好。

接著執行：

PowerShell

```
Get-Item .\job-1-old\job-1.html | Select-Object FullName, Length, LastWriteTime
```

這會告訴我們：

- 檔案大小
- 最後修改時間

這個時間非常重要。

第二步：不要覆蓋任何東西，先做一份備份

如果 `job-1-old\job-1.html` 還在，執行：

PowerShell

```
Copy-Item .\job-1-old\job-1.html .\job-1\job-1-可能的正確版本.html
```

這樣即使後面操作錯誤，原始檔還在。

第三步：檢查 Git 的 6e157b0

我們也可以把 9/23 凌晨 1 點以前最後一個 Git 版本取出來比較：

PowerShell

```
git show 6e157b0:job-1/job-1.html > .\job-1\job-1-20260920版本.html
```

現在 `job-1` 會有：

```
job-1
├─ job-1.html
├─ job-1-可能的正確版本.html
└─ job-1-20260920版本.html
```

先不要改名，也不要覆蓋 `job-1.html`。

★ 我們真正要找的其實可能是「未 Commit 的版本」

因為你的紀錄很有意思：

```
9/20 15:08 → feat:標題
↓
9/23 02:07 → feat:job1
```

↓
9/23 02:09 → feat:job1

你說：

正確版本是 2026/9/23 1點前

而 Git 在 **2:07**、**2:09** 才出現 `job-1` 的 Commit。

因此，如果你在 **1:00~2:00** 左右曾經編輯過 `job-1.html`，但尚未 **Commit**，那個版本不一定存在於 GitHub 的 Commit 歷史裡。

但它可能還存在於：

- `job-1-old`
- Windows 資源回收筒
- VS Code 本地歷史 / Timeline
- VS Code Auto Save / 備份
- Git reflog（如果曾經 commit 過）
- 其他備份

現在最重要的就是 `job-1-old`

請你先執行：

PowerShell

```
Get-Item .\job-1-old\job-1.html | Select-Object FullName, Length, LastWriteTime
```

把結果貼給我。

不要再做任何 `pull`、`push`、`reset` 或刪除動作。

如果 `job-1-old\job-1.html` 的 `LastWriteTime` 是你說的 **2026/9/23 凌晨 1 點以前**，那我們很可能已經找到你原本的正確版本了。

我剛才是直接把github的html內容貼到本機的html內容後直接pull
所以本機只有一個貼錯的job-1.html,沒有另外備份

了解。這樣情況就很明確了：

原本正確的 `job-1.html` 沒有另外備份，而且你後來把錯誤內容貼回本機，再執行了 Git 操作。

但先不要放棄。你目前仍然有一個很重要的救援機會：Git 的歷史與 `reflog`。

你之前查到：

```
060d8e1 | 2026-09-23 02:07:19 | feat:job1
ec2f821 | 2026-09-23 02:09:51 | feat:job1
6e157b0 | 2026-09-20 15:08:22 | feat:標題
0d7b8a4 | 2026-09-20 09:33:30 | feat:大綱
```

其中最值得先查的是 **060d8e1** 和 **ec2f821**。

現在請「不要再 pull」

也不要：

PowerShell

```
git reset --hard
```

也不要：

PowerShell

```
git push
```

我們先把 Git 裡可能存在的版本全部找出來。

第一步：查看兩個 Commit 裡的 `job-1.html`

先輸入：

PowerShell

```
git show 060d8e1:job-1/job-1.html
```

這會直接顯示 **060d8e1** 當時保存的 `job-1.html`。

然後再輸入：

PowerShell

```
git show ec2f821:job-1/job-1.html
```

如果其中一個就是你原本正確的版本

不要直接複製貼回目前的 `job-1.html` 。

先把它另存成：

PowerShell

```
git show 060d8e1:job-1/job-1.html > .\job-1\job-1-救援版.html
```

或者如果是 `ec2f821`：

PowerShell

```
git show ec2f821:job-1/job-1.html > .\job-1\job-1-救援版.html
```

這樣我們就保留了救回來的版本。

第二步：如果 060d8e1 / ec2f821 都不是

再查 Git 的「指標移動紀錄」：

PowerShell

```
git reflog --all --date=iso
```

這個非常重要。

`reflog` 有可能找到曾經存在於本機 Git 的 Commit，即使現在 `main` 已經不指向它。

第三步：搜尋 Git 裡所有可能的舊版本

執行：

PowerShell

```
git fsck --full --no-reflogs --unreachable
```

如果出現：

```
unreachable commit xxxxxxxx
```

不要刪任何東西。

我們可以再逐個檢查那些 Commit 裡的：

```
job-1/job-1.html
```

但是有一個關鍵限制

如果你說的「正確版本」：

在 2026/9/23 凌晨 1 點以前存在於本機，但從來沒有 Commit 到 Git

那麼 Git 不一定有辦法找回它。

因為 Git 只能可靠保存「已經加入並 Commit 的版本」。

這種情況我們就要改找：

VS Code 的 Timeline / Local History / Windows 檔案歷程 / 暫存檔。

★ 所以現在最重要的第一步

請你只執行這兩個：

PowerShell

```
git show 060d8e1:job-1/job-1.html
```

以及：

PowerShell

```
git show ec2f821:job-1/job-1.html
```

不要貼全部內容給我也可以。

你只要告訴我：

060d8e1 是不是正確版本？

ec2f821 是不是正確版本？

如果兩個都不是，我們就立即進入 **reflog** → **unreachable commit** → **VS Code 本地歷史** 的救援流程。

而且你目前的情況，還不要刪除任何 Git 資料或重新 clone Repository。這些東西可能正是我們最後可以救回舊版本的依據。

正確版本在 2026/9/23 凌晨 1 點以前在github上,不在本機

了解，這個資訊非常重要。那麼我們現在要找的不是「本機舊檔」，而是：

GitHub 在 2026/9/23 凌晨 1:00 以前曾經存在的 `job-1.html`。

你目前的 Git 歷史顯示，`job-1.html` 的第一筆紀錄是：

```
060d8e1 | 2026-09-23 02:07:19 | feat:job1
ec2f821 | 2026-09-23 02:09:51 | feat:job1
```

這反而提供了一個重要線索：

如果你確定凌晨 1 點以前 GitHub 已經有正確的 `job-1.html`

那麼正常的 Git Commit 歷史裡應該還有一個更早的版本，除非：

1. 之前的 Commit 被 `rebase` / `force push` 改寫；
2. `job-1.html` 曾經存在於另一個分支；
3. GitHub 上的檔案是在某個後來被改寫的 Commit 中加入；
4. 你看到的「凌晨 1 點以前」其實是 GitHub 檔案內容的更新時間，而不是 Commit 時間。

現在不要再修改 GitHub

尤其不要再：

PowerShell

```
git push
```

我們先找 GitHub 歷史。

請直接打開這個 GitHub 頁面

[查看 job-1.html 的 GitHub Commit 歷史](#)

GitHub 上應該會列出這個檔案過去的 Commit。

你要找的是：

```
2026/09/23 01:00 前
```

的版本。

如果 GitHub 顯示更早的 Commit

例如看到：

```
2026-09-23 00:xx  
feat: xxx
```

點進那一筆 Commit。

然後找到：

```
job-1/job-1.html
```

再點進去。

這樣就可以看到當時的完整 HTML。

更重要：如果 GitHub 現在只剩 02:07 和 02:09

如果 GitHub 的歷史現在只剩：

```
02:09 ec2f821  
02:07 060d8e1  
09/20 6e157b0
```

而完全沒有 01:00 前的 `job-1.html`，那就表示那個舊版本可能已經被 Git 歷史改寫掉。

這時不要再用 `git pull` 解決。

我們要找的是 GitHub 是否還保留：

- 其他 branch
 - Pull Request 的 head commit
 - force-push 前的 commit
 - GitHub 的其他 refs
-

我可以直接幫你查這件事

你可以把下面這個頁面的畫面截圖給我：

job-1.html 的 GitHub 歷史

尤其是 **2026/9/23 凌晨 1 點以前的部分**。

如果你看不到更早版本，我們下一步就查：

```
GitHub branches
↓
Pull Requests
↓
reflog / unreachable commits
```

目前最重要的一句話

不要再對 GitHub 做任何 push。

因為我們現在是在做「舊版本救援」，不是一般的同步。只要舊 Commit 或其他 GitHub ref 還存在，就有機會把你凌晨 1 點以前的正確 `job-1.html` 找回來。

ChatGPT 可能會出錯。請查核重要資訊。